

Développement front

MMI 2 – CM#6 S3



Danielo **JEAN-LOUIS**

Javascript

- Langage de programmation orienté pour le web
- Créé en 1995 par Brendan Eich
- Langage différent du Java malgré son nom

Javascript

- Utilise .js comme extension de fichier
- Permet notamment d'ajouter de l'interaction sur les sites
 - Présent sur quasiment tous les sites de nos jours

Javascript

- Le code est lu de haut en bas par le navigateur
- Reprend les concepts d'autres langages : variables, fonctions... avec une autre syntaxe

Javascript

- Trois façons de charger un fichier js :
 - Dans une balise `<script>`
 - Attribut html : à bannir
 - Fichier externe
 - Préférer cette méthode pour des questions de lisibilité

Il est possible de combiner les méthodes au sein du même projet

Javascript

- Le point-virgule est facultatif (sauf rares cas)
- Langage faiblement typé :
 - Un entier peut devenir une chaîne de caractères ! Et vice-versa.

Langage extrêmement permissif. Produisez un code rigoureux !

Chargement des scripts - balise



```
<body>
  <!-- [ ... ] -->
  <script>
    /* Code javascript */
  </script>
</body>
```

Pour éviter des problèmes, la balise script se place à la fin de la balise <body>

Note : les commentaires sont accessibles à l'utilisateur final

Chargement des scripts – fichier externe



```
<!DOCTYPE html>
<html lang="fr">
<head>
    <!-- [ ... ] -->
    <script src="chemin-vers-fichier.js" defer></script>
</head>
<!-- [ ... ] -->
</html>
```

Notre code javascript est contenu dans un fichier externe puis chargé dans un fichier HTML.
Le chemin peut être relatif ou absolu.

Attribut "defer"

```
<!-- [ ... ] -->  
<script src="chemin-vers-fichier.js" defer></script>  
/head>
```

Source(s) :

- <https://developer.mozilla.org/fr/docs/Web/HTML/Element/script#attr-defer>

Attribut "defer"

- Attribut propre à la balise `<script>`
- Indique au navigateur de charger le script **après chargement** des balises HTML
 - Limite les risques de bugs si le script est chargé avant les balises HTML
- Ne fonctionne qu'avec l'attribut "src"
- **Pensez à le mettre**

Source(s) :

- <https://developer.mozilla.org/fr/docs/Web/HTML/Element/script#attr-defer>

Attribut "defer"

- Attribut propre à la balise `<script>`
- Indique au navigateur de charger le script **après chargement** des balises HTML
 - Limite les risques de bugs si le script est chargé avant les balises HTML
- Ne fonctionne qu'avec l'attribut "src"
- **Pensez à le mettre**

Source(s) :

- <https://developer.mozilla.org/fr/docs/Web/HTML/Element/script#attr-defer>

Variables

- Trois mots clés :
 - var : désuet. Crée des effets de bord
 - let : Crée une variable dont la valeur peut changer
 - const : Crée une constante. La valeur ne change pas

Source(s) :

- <https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Statements/const>
- <https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Statements/let>

Variables

- Préférez un nom clair et descriptif
 - Par convention, on utilise la camelCase.
Ex : ageEtudiant, nbreEtudiants

Source(s) :

- <https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Statements/const>
- <https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Statements/let>

Variables



```
const jeSuisUneConstante = "Je ne changerai pas";  
let jeSuisUneVariable = "Je peux changer";  
  
// A éviter - mot-clé var  
var jeSuisUneSyntaxeDesuete = "A éviter";
```

Source(s) :

- <https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Statements/const>
- <https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Statements/let>

Variables - Règles

- Ne pas mettre d'espaces
- Ne pas commencer par un nombre
 - Mais peut contenir un nombre
- Ne pas mettre de tirets ou autres caractères spéciaux (underscore possible)
- Éviter les accents

Source(s) :

- <https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Statements/const>
- <https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Statements/let>

Fonctions

- Permettent de réutiliser le code
 - Évite de se répéter
- Permettent de mieux séparer le code
 - Meilleure lisibilité
- Créent un contexte qui leur est propre
 - Toute variable créée à l'intérieur n'est pas accessible à l'extérieur

Source(s) :

- <https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Operators/function>

Fonctions



JS

```
// Déclaration de la fonction
const maFonction = (parametre) => {
    let maVariable = "Hello";

    return maVariable;
}

// Appel de la fonction avec paramètre
maFonction("BUT MMI");
```

Source(s) :

- <https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Operators/function>

Fonctions – mot-clé "return"

- Permet d'assigner le "résultat" d'une fonction dans une variable
- Présence multiple de return possible dans une fonction
 - Une fonction ne peut retourner **qu'un seul élément à la fois**

Source(s) :

- <https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Operators/function>

Fonctions – mot-clé "return"

- Utilisable **uniquement** dans une fonction
- Met fin à l'exécution d'une fonction
 - Toute ligne après le mot-clé "return" et au même niveau ne sera pas exécutée
- Disponible dans d'autres langages de programmation

Source(s) :

- <https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Operators/function>

Fonctions – mot-clé "return"



JS

```
const addition = (parametre1, parametre2) => {  
    return parametre1 + parametre2;  
    console.log("ne sera jamais exécuté");  
}
```

Cette ligne est après un "return" et au même niveau, elle ne sera jamais exécutée

Source(s) :

- <https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Operators/function>

Fonctions – Valeurs par défaut

- Appliquent une valeur aux paramètres non définis



JS

```
function multiplication(a, b = 1) {  
    return a * b;  
}
```

Si l'argument "b" n'est pas défini lors de l'appel de la fonction. Alors sa valeur sera égale à l'entier "1"

Source(s) :

- <https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Operators/function>

Autres instructions

- if...else : exécute une instruction si la condition est remplie
- For : Exécute des instructions tant que la condition de bouclage est vraie

Source(s) :

- <https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Statements/if...else>

console.log

- Equivalent js de print() en php
- Affiche le résultat dans la console du navigateur onglet “Console”
- Très utile pour débbugger son code
 - **Servez-vous en**

Source(s) :

- https://developer.mozilla.org/fr/docs/Web/API/console/log_static

Manipulation de la page web

- Permet d'éditer / supprimer ou ajouter des éléments HTML sur la page
- Possibilité de récupérer un ou des éléments via des méthodes dédiées comme “querySelector()”

Source(s) :

- <https://developer.mozilla.org/fr/docs/Web/API/Document/querySelector>

Manipulation de la page web - `querySelector`

- Méthode permettant de récupérer un élément sur la page courante
- Prend en paramètre un sélecteur CSS
- Retourne le **premier** élément trouvé



JS

```
const monElement = document.querySelector(".element");
```

Source(s) :

- <https://developer.mozilla.org/fr/docs/Web/API/Document/querySelector>

Manipulation de la page web - querySelector

- Méthode coûteuse en performance
 - Si possible évitez de l'appeler dans une boucle

Source(s) :

- <https://developer.mozilla.org/fr/docs/Web/API/Document/querySelector>

Démo

Gestion des évènements

- Utilisation de la méthode “addEventListener” permettant d’écouter un évènement
 - Clic, focus, survol...
- Appel d’une fonction dépendamment d’un évènement qui a eu lieu

Source(s) :

- <https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Statements/if...else>

Questions ?

